Experimental Platform

The experiments were performed using a quadrotor running the PX4 flight controller software stack. The vehicle used was a Modalai Sentinel Development Drone with a Mircohard Modem Carrier Board for wireless/RF communication. The Sentinel vehicle features a sensor suite consisting of 5 OV7251 global shutter VGA cameras (2 front stereo, 2 rear stereo and 1 front-mounted downward-facing tracking camera), 1 Sony IMX214 4k Hi-res camera, 2 TDK ICM-42688-P IMUs, 2 TDK ICP-10100 barometers and 1 Ublox NEO-M8N GPS/Magnetometer module (1). The vehicle weighs 1.396 kg (with battery), has a payload capacity of 1 kg and a thrust to weight ratio of 2.32. The battery used was a MaxAmps 3 Cell Lithium-Polymer (LiPo) with a capacity of 5450 mAh and a nominal voltage of 11.1 V. The vehicle frame is a symmetric X configuration with a length and width of 384 mm and a height of 133 mm (187 mm with GPS module fully extended). The vehicle is equipped four Holybro 2216-880kv motors and four Master Airscrew Glass Fiber R Composite propellers with 10 inch diameter and 4.5 inch pitch.

The quadrotor is equipped with the Modalai VOXL 2 onboard computer and autopilot built on the Qualcomm QRB5165 CPU. The VOXL 2 onboard computer was flashed with Modalai's voxl-suite version 1.1.2, consisting of a system image running a Linux OS (Ubuntu 18.04 kernel 4.19.125) and a software suite consisting of services and libraries enabling running and communicating with an onboard flight controller. The VOXL 2 directly runs a custom branch of the PX4 autopilot firmware as the flight controller on the onboard digital signal processor and communicates to the flight controller via UART connections. The customizations of the PX4 branch amounts to sensor and electronic speed controller (ESC) specific drivers and board specific parameters, but the source code of the flight stack (e.g. control architecture, navigation modules, etc...) is identical to the version 1.14 branch from which it was forked.

Outside of vehicle specific tuning that was performed by the manufacturer (e.g. controller PID gains, ESC parameters) the only parameters modified for these experiments were those setting up the sensor fusion for the flight controller's state estimation module, PX4's built-in Extended Kalman Filter (EKF) implementation. As the experiments were conducted indoors and a reliable external-vision system was available for localization, the EKF was configured to ignore sensor input from the GPS, magnetometer and barometer as to avoid inaccuracies introduced by the flight environment. The EKF was instead configured to only fuse sensor input from the IMU (a combined accelerometer and gyroscope) and the external vision system when generating state estimates.

An Optitrack motion capture system consisting of 59 Prime 41 ethernet cameras and a Windows 10 PC running the Optitrack Motive application was used as the external vision system. Position and quaternion orientation measurements of the vehicle in the global "mocap" frame were streamed from the Motive application to a client application on the ground station computer (a laptop running Ubuntu 20.04). In order for external vision measurements to be fused by PX4's EKF, they must be in a compatible reference frame and include valid velocity measurements and covariances. To accommodate PX4's requirements for external vision measurements, or vehicle visual odometry, the position and orientation measurements from the Motive application were transformed into the vehicle local North-East-Down (NED) frame, aligned with the vehicle Front-Right-Down (FRD), or body frame, from vehicle start-up. The initial position and orientation are captured prior to flight in order to establish the global to local frame transformations and rotations, which are then applied for each

subsequent measurement to place each measurement received in the vehicle local NED frame. The transformed measurements are then passed through a filtering application, which performs a Random Sample Consensus (RANSAC) algorithm to generate linear velocity estimates from the best-fit line through a sliding window of the past and current positions. The complete measurement is then passed through an Extended Kalman Filter implementation to provide a filtered state odometry estimate for the autopilot. A custom service deployed on the onboard computer receives the odometry estimates from the ground-station computer, converts the local frame measurements to a synthetic GPS measurement and passes both local and global state estimates to the PX4 flight controller over UART via the Mavlink messaging protocol.

The VOXL SDK software suite contains a built-in service that employs a proprietary SLAM algorithm which fuses onboard camera sensor data with onboard IMU sensor data to generate a visual inertial odometry (VIO) measurement (local position, velocity, orientation and angular velocity). To avoid the inherent drift over time of measurements associated with this method of state estimation (those not tied to a fixed global frame but rather only the inertial frame), the decision was made to drop all outgoing VIO measurements and use the service for vehicle localization only as a failsafe should the motion capture system crash or stall. There were no instances of motion capture system failure during the course of the experiments and as such the onboard VIO system was not employed.

Flight Controller

The flight controller used for this experiment was a custom implementation of the version 1.14.0-2.0.79 branch of the PX4 Autopilot firmware. Modifications of the firmware were limited to vehicle-specific drivers for the ESC's, onboard and external sensors, and gain tuning performed by the manufacturer. The remaining portions of the flight stack (e.g. the EKF state estimation module, the controller modules, the navigator module) are unchanged from the standard PX4 release. All modules within the flight stack intercommunicate via a uORB messaging bus, which implements the uORB asynchronous publisher/subscriber messaging application programming interface (API) (2). Modules publish and subscribe to uORB topics in order to facilitate inter-thread and inter-process communication. External communication with the PX4 flight controller can be accomplished either through direct subscription/publication to the internal uORB by a middleware using the same serialization protocol (eProsima Mirco XRCE-DDS) such as the robot operating system (ROS) software or by using the Mavlink serialization protocol to send messages to PX4's mavlink module over UART, which are then parsed and republished to the appropriate uORB topics. For these experiments, the Mavlink messaging protocol was chosen as the means of external communication with the flight controller.

The PX4 flight controller implements a cascaded PID control architecture to perform control over position, attitude and angular rates of the vehicle. This consists of a position controller, attitude controller, rate controller and control allocator (or mixer). The position controller is made up of a proportional controller that acts on position error to produce desired velocity, a PID controller that acts on velocity error to produce a desired acceleration, and a series of equations that convert the desired acceleration to a desired attitude and desire thrust in the vehicle body frame. The desired attitude generated from the position controller is published to the attitude controller, which acts as a proportional controller over the attitude error and outputs desired angular rate to the rate controller. The rate controller acts as a PID controller over the angular rate error and outputs desired body frame

torque to the mixer/control allocator. Desired body frame thrust is passed directly to the mixer from the position controller. The mixer then maps desired thrust and torque to direct actuator commands (motor speed) via PWM messages to the ESC.

Each layer of the control loop is rate limited by the sensor topic needed to define an error term. The position controller subscribes to the vehicle local position topic (published by the vehicle EKF at 100 Hz) and the attitude controller and rate controller subscribe to the filtered imu data (output from the IMU drivers at 250 Hz). As such, the maximum loop rates of the position, attitude and rate controllers are 100 Hz, 250 Hz and 250 Hz respectively.

Each layer of PX4's control architecture can be bypassed by either sending target position, attitude or rate setpoints or by enabling different flight modes. When position, attitude or rate control flight modes are enabled, stick input from the RC controller is used as the source of desired trajectory, attitude or rate setpoints. When PX4's offboard flight mode is enabled the PX4 accepts user generated setpoints.

For these experiments, each controller under test received a desired trajectory, calculated a position and velocity error from the current state estimate and output a desired acceleration. The desired acceleration and control mode was sent to PX4 via Mavlink messages. The desired accelerations were then converted by PX4's position control module to desired thrust and attitude which were sent directly to the attitude controller, thus bypassing PX4's default proportional position error controller and PID velocity error controller. The onboard controllers under test were run at 100 Hz to match the maximum loop rate of the position control module.

Ground Station / Communication Pipeline

The communication pipeline with the vehicle consisted of three computing endpoints: a desktop computer running the Motive application, a laptop acting as the ground-station and the vehicle onboard computer. The Motive PC running Windows 10 OS was connected directly to the network switch aggregating the camera feed via a 10 Gbps data link and was connected to the ground-station computer over a Local Area Network (LAN) via ethernet. The latency of data transfer from the Optitrack system and the ground-station computer was measured using two metrics: system latency and transit latency. The system latency was measured as the time between the mid-exposure camera timestamp (when the actual image was captured) and after the frame data was finished being processed and packaged to stream. Transit latency was measured as the time between the frame data being streamed out of the Windows PC running Motive and the time it was received by the client application on the ground-station. The measured system latency for this set-up was 7.61 milliseconds on average and the measured transit latency was 0.49 milliseconds on average. For reference, the cameras were run at a frame rate 120 Hz, or 0.083 milliseconds between frames. The Optitrack Natnet SDK software was used for data transfer between the Motive application on the Windows PC and the client application, which provides an API using User Datagram Protocol (UDP) for network communication (3).

The client application receiving the motion capture pose data then corrected incoming data to the vehicle local frame and immediately republished it to a filtering application that generate linear and angular velocity measurements from the incoming pose data and streamed it to the vehicle. A Random

Sample Consensus (RANSAC) algorithm was developed to perform line-fitting in each dimension over a sliding window of the ten latest received local position measurements. An outlier threshold was defined by the user and the slope of the line with the greatest number of inliers was chosen as the linear velocity estimate. If more than one line had the same number of inliers, the line with the lowest mean squared error for the distances of the individual positions from the lines was chosen. An Extended Kalman Filter (EKF) was developed in order to generate a filtered complete state estimate (position, attitude, velocity and angular velocity) from an incomplete and potentially noisy measurement of position, quaternion attitude and linear velocity. After the filtered state estimate was generated, it was streamed to a client service running onboard the vehicle. The data filtering application on start-up also communicates with a clock-syncing service running onboard the vehicle so that all outgoing data packets can be converted from ground-station to vehicle compatible timestamps.

All intercommunication between applications on the ground-station and communication between the ground-station was accomplished using the ZeroMQ (or ZMQ) networking software library and it's C++ binding, which provides broker-less asynchronous messaging via socket objects and several messaging patterns and transport protocols (4). Communication between the motion capture data client application and the data filtering application was accomplished using ZMQ sockets in a publisher/subscriber architecture communicating over the Inter-Process Communication (IPC) protocol, which is an OS-dependent protocol that on UNIX systems is akin to reading/writing from POSIX pipes. Communication between the data filtering application and vehicle was accomplished using ZMQ sockets in a request/reply architecture for communicating with the onboard time synching service and a publisher/subscriber architecture for communicating with the onboard odometry client, both using the Transmissions Control Protocol (TCP) for communicating over the network. Ground-station to vehicle networking was accomplished by establishing a LAN with Microhard Pico Ethernet Motherboard housing a Pico series Multiple Input Multiple Output (MIMO) digital data link (pMDDL2450) radio. The ground-station radio establishes a point-to-point connection with a pMDDL2450 radio onboard the vehicle housed in the Microhard Modem Carrier Board, an add-on to the default Sentinel vehicle configuration enabling wireless RF communication.

Latencies associated with the ground-station communication consists of the processing time of the motion capture data client application, the transit latency over IPC between the motion capture client and the data filtering application, the processing time of the data filtering application and the transit latency between the ground-station application and the vehicle client application. Average latencies for each segment were evaluated as follows:

Application	Processing Latency (ms)	Transmission Latency (ms)	Transmission Protocol
Motion Capture Server	7.611565	0.4904318	UDP
Motion Capture Client	0.000028564	0.114315	IPC
Data Filtering Application	3.77939	1.888709	TCP
Vehicle Mocap Client	-	0.078472	POSIX Pipes

Total System Latency	11.39098	2.4934558	-

Software Stack

The software applications running on the ground-station included the data filtering and transmitting application (the "odometry estimator") and the Optitrack data client application. Both were written using standard C and C++ libraries. The C++ library Eigen version 3.3.7 was used for linear algebra applications and the ZeroMQ C library version 4.3.2 and its C++ binding CPPZMQ version 4.10.0 were used for networking for both applications. The NatNet SDK version 4.0 was used for interfacing with the Motive application in the Optitrack client application.

There were three software packages that needed to be created and run onboard the vehicle in order to run the experiment: a ground-station to vehicle clock synchronization service, a client service subscribing to the odometry data streamed from the ground-station and republishing it to the autopilot via the Mavlink protocol, and a service implementing the controllers under test and sending offboard flight commands to the autopilot over Mavlink. All three applications were written in C and C++ and used ZeroMQ and CPPZMQ for networking with the ground-station. The C++ library GeographicLib version 2.3 was used in the vehicle odometry client application for converting local coordinates into a global coordinate frame to set a global origin and enable a stream of synthetic GPS data from the incoming motion capture position data. The vehicle manufacture provides several open-source software libraries as part of the VOXL SDK releases to facilitate ease of communication between onboard services. The libmodal-pipe library was used to implement POSIX pipes for onboard inter-process communication and the libmodal-json library was used to implement json configuration files for each of the onboard services. The Mavlink C library was used for serialization of outgoing trajectory setpoints and odometry messages to the PX4 autopilot.

Methodology

The experiments were conducted in an indoor flight enclosure consisting of a 25.16 m by 11.81 m rectangular grid of 59 cameras approximately equally spaced at heights of 3.9 m and 8.3 m above the ground plane. The motion capture frame origin was located at the geometric center of the camera grid, with the x-axis aligned parallel to the shorter faces of the grid and the z-axis aligned parallel to the longer faces of the grid and the y-axis pointing up perpendicular to the ground plane. For each flight, the vehicle was positioned at the origin such that the vehicle local North-East-Down (NED) frame mapped to the motion capture system global frame as follows: Local North axis mapped to the motion capture positive x axis, Local East axis mapped to the motion capture positive z axis, Local Down axis mapped to the motion capture negative y axis.

A water bottle was attached with a 0.083 meter (3.25 inch) length of nylon line to the vehicle's right landing skid. Two K Tool International (KTI) 42-inch Belt Drive Drum Fans with an air flow capacity

of 14800 cubic feet per minute at 537 rpm were placed at positions 1.83 meters south and 7.32 meters west of the origin with respect to the vehicle local NED frame.

The desired trajectory for the experiment was broken into five stages: takeoff, initial hover, figure eight cycles, final hover, and landing. The vehicle was given 5 seconds to takeoff from an armed, grounded position at the origin and reach the hover position at 0.5 meters above the origin, then given 10 seconds to execute a hover hold at that position. After completing the initial hover sequence, the vehicle was given two minutes to perform four cycles of an oscillating-height figure eight pattern (30 seconds per cycle), followed by a final hover hold at 0.5 meters above the origin for 10 seconds. After completing the final hover, the vehicle was given 5 seconds to complete a two-stage landing. The first stage of the landing consisted of a vertical descent to a position 0.25 meters above the origin and the second stage consisted of a slanted descent to a position 0.25 meters in the local frame west direction on the ground plane. The slanted descent was required in order to tip over the water bottle attached to the vehicle landing skip, to prevent the upright water bottle from tipping over the vehicle in the case of a vertical landing. The duration of each experiment was

The figure eight portion of the desired trajectory was generated using a parameterized version of Bernoulli's lemniscate in the horizontal plane, with a sinusoidal function defining the oscillating height of the trajectory. The equations used were as follows:

$$x = -\frac{r_1 cos\theta \, sin\theta}{\sin^2 \theta + 1}$$

^ in case this equation isn't displayed properly it is:

X = - (r1 * cos(theta) * sin(theta)) / (sin^2(theta) + 1)

$$y = \frac{r2 \cdot cos\theta}{\sin^2 \theta + 1}$$

^ in case this equation isn't displayed properly it is:

y = (r2 * cos(theta)) / (sin^2(theta) + 1)

$$z = (flight\ altitude) + r_3 \cdot cos\theta$$

^ in case this equation isn't displayed properly it is:

z = (flight altitude) + r3 * cos(theta)

Values of r1, r2 and r3 were set to 1.5, 5.0 and 0.25 meters respectively. The flight altitude was set to - 0.5 meters. Theta is defined as the angle sweeping from $\frac{\pi}{2}$ (negative pi over 2, in case this doesn't display when I send it) to $\frac{3\pi}{2}$(positive 3pi over 2, again in case the equation doesn't display properly). This resulted in a trajectory spanning from –0.53 to 0.53 meters in the x direction (aligned with North axis of vehicle local NED frame), from –5.0 to 5.0 meters in the y direction (aligned with East axis of vehicle local NED frame) and from –0.25 to –0.75 meters in the z direction (aligned with Down axis of vehicle local NED frame). Note, when the z-axis is aligned with vehicle local NED Down axis, an altitude above the ground plane maps to a negative value.

Since the equations used to generate the trajectory are continuously differentiable, desired velocity and acceleration were generated as part of the planned trajectory. The angular change with respect to time ($\frac{d\theta}{dt}$) is held fixed at 0.0021 radians per second by the requirement to complete each loop of the figure-eight portion of the trajectory in 30 seconds. The equations for desired velocity and acceleration are as follows:

$$v_x = \frac{\frac{d\theta}{dt} r_1 (\sin^4 \theta + (\sin^2 \theta - 1) \cdot \cos^2 \theta)}{(\sin^2 \theta + 1)^2}$$

^ in case this equation isn't displayed properly it is:

vx = (dtheta/dt * r1 * (sin^4(theta) + sin^2(theta) + ((sin^2(theta) -1) * cos^2(theta))) / (sin^2(theta) + 1)^2

$$v_y = -\frac{\frac{d\theta}{dt} r_2 sin\theta (\sin^2 \theta + 2\cos^2 \theta + 1)}{(\sin^2 \theta + 1)^2}$$

^ in case this equation isn't displayed properly it is:

vy = - (dtheta/dt * r2 * sin(theta) * (sin^2(theta) + 2* cos^2(theta) + 1) / (sin^2(theta) + 1)^2

$$v_z = -\frac{d\theta}{dt} \cdot r_3 \cdot sin\theta$$

^ in case this equation isn't displayed properly it is:

vz = - (dtheta/dt) * r3 * sin(theta)

$$a_x = -\frac{\left(\frac{d\theta^2}{dt}\, 8\, r_1 sin\theta cos\theta \cdot \left((3\cos 2\,\theta) + 7\right)\right)}{(\cos 2\,\theta - 3)^3}$$

^ in case this equation isn't displayed properly it is:

ax = - ((dtheta/dt)^2 * 8.0 * r1 * cos(theta) * sin(theta) *((3 * cos(2*theta) + 7)) / (cos(2*theta) - 3)^3

$$a_y = -\frac{\left(\frac{d\theta^2}{dt}\, r_2 cos\theta \cdot \left((44\cos 2\,\theta) + \cos 4\,\theta - 21\right)\right)}{(\cos 2\,\theta - 3)^3}$$

^ in case this equation isn't displayed properly it is:

ay = ((dtheta/dt)^2 * r2 * cos(theta) * ((44 * cos(2*theta) + cos(4*theta) - 21)) / (cos(2*theta) - 3)^3

$$a_z = -r_3 \frac{d\theta^2}{dt}\, cos\theta$$

^ in case this equation isn't displayed properly it is:

az = - r3 * (dtheta/dt)^2 * cos(theta)

The flight altitude for the desired trajectory was set such that during portions of the figure-eight trajectory, while the vehicle was descending from the hover flight altitude to the low point of the trajectory, the bottle would be dragging along the ground. This was an intentional aspect of the experiment, meant to induce additional uncertainties for the controller to adapt to and overcome by creating an additional drag force acting against the vehicle and inducing more exaggerated oscillation of the bottle as it was lifted off of the ground during each cycle of the figure-eight pattern.

Flight testing took place in doors with a motion capture system using infrared lighting and reflective markers on the vehicle. Flight tests took place between in the early afternoon (between 9:00 a.m and 10:00 a.m local time) to limit the amount of direct sunlight coming through the building windows to prevent extraneous reflections interfering with the accuracy of the motion capture

estimates. All reflective materials were removed from flight area, with the exception of the metal box-fans required to run the experiments. Additional reflections, such as those detected from cameras around the flight area were masked within the motion capture system. All flights were conducted back-to-back to reduce the amount of environmental variation between tests. As no issues arose during the course of testing, each controller was tested successfully and only once. Environmental conditions during testing consisted of an average temperature of 37.3 degrees Celsius, an average air pressure of 101822.12 Pascals and an average air density of 1.2098 kilograms per cubic meter.

The motion capture system as well as the vehicle onboard sensors (IMU, barometer, magnetometer) were calibrated using standard PX4 and Optitrack calibration procedures prior to running the experiments. The vehicle and ground-station clocks were synchronized by use of a common Network Time Protocol (NTP) server. All timestamps generated on the ground-station were converted with respect to the vehicle monotonic clock (used by the autopilot) by the time-synchronization application written for this experiment. Time synchronization between the ground-station and the Motive PC was achieved by use of a built-in client class within the Natnet SDK which employs Cristian's algorithm (6) to perform synchronization with the Natnet server generated by the Motive application. Data logging was performed onboard the vehicle computer via csv file-writing functions within the offboard controller program as well as PX4's built-in ULog logging system.

Repeatability and Validation

This experiment can be reproduced using any quadrotor running a PX4 (or functionally equivalent) autopilot provided there is reliable, low latency communication to and from the autopilot and an onboard computer and between the onboard computer and a ground-station computer. A motion capture system is not required as long as there is a reliable, accurate, real-time system of sensor input to provide vehicle localization required to generate error estimates between the actual and desired position and velocity of the vehicle. The ability to input desired acceleration setpoints to the autopilot's control architecture is required. In the case of the PX4 autopilot, this can be achieved using the Mavlink protocol's built-in trajectory setpoint message or via direct publication to the autopilot's offboard mode and trajectory setpoint uORB topics by any program using a compatible middleware, such as ROS2.

References

1. https://docs.modalai.com/sentinel-functional-description/
2. https://docs.px4.io/main/en/middleware/uorb.html
3. https://optitrack.com/software/natnet-sdk/
4. https://zeromq.org/get-started/
5. https://gitlab.com/voxl-public/voxl-sdk

6. *Cristian, Flaviu (1989), "Probabilistic clock synchronization" (PDF), Distributed Computing, **3** (3), Springer: 146–158, doi:10.1007/BF01784024, S2CID 3170166*

^This reference is just pulled from the wikipedia article for Cristian's algoritim